

Visual Compliance Inspector: Architecture Blueprint

NVIDIA NemoClaw Security Envelope, Cosmos 3 Reasoner and Multi-Channel Chat Orchestration

1. Executive Overview

This architectural plan details the transformation of the Visual Compliance Inspector into a secure ambient AI agent. By encapsulating autonomous agent capabilities inside NVIDIA NemoClaw and NVIDIA OpenShell, users can interact with the system via natural language and send video/document attachments across enterprise chat channels (WhatsApp, Telegram, Slack) while enforcing strict container sandboxing, data privacy, and deterministic audit verification.

2. Component Stack and Roles

Component	Technology	Primary Role and Scope
Security Envelope	NVIDIA NemoClaw (OpenShell)	Kernel sandboxing, policy router, egress firewall, process isolation
Agent Gateway	OpenClaw Framework	Multi-channel chat bridging (WhatsApp, Slack, Telegram), tool dispatch
Agent Intelligence	Nemotron-3.5 Lightning	Intent extraction, tool calling, natural language audit synthesis
Vision Reasoner	Cosmos 3 Reasoner NIM	Temporal video ingestion (1 fps), spatial reasoning, timestamp detection
Audit Engine	Deterministic Validator	Guaranteed sequence validation (start_sec < end_sec), zero hallucinations
Presentation UI	Streamlit and Chat Responses	Interactive visual dashboard and direct mobile chat summaries

3. Can NemoClaw and OpenClaw Handle WhatsApp / Chat Interfaces?

- YES. In fact, chat connectivity is one of the core strengths of the OpenClaw platform. OpenClaw functions as a self-hosted messaging gateway that connects everyday apps directly to local AI agents:
- > Native Messaging Bridge: OpenClaw natively supports WhatsApp (via WhatsApp Business Cloud API or Twilio), Telegram (via BotFather token), Slack, Microsoft Teams, and Discord.
 - > Document and Media Ingestion: When a user uploads an MP4 video or JSON SOP in WhatsApp, OpenClaw automatically downloads the attachment into a sandboxed staging directory.
 - > Autonomous Dispatch: Nemotron-3.5 receives the user message (e.g. 'Audit this video against SOP v2') along with the file path, invokes the compliance tool, and executes the pipeline.
 - > Bidirectional Response: Once analysis finishes, OpenClaw sends a formatted compliance verdict (PASS / FAIL, timestamp evidence) and generated PDF reports directly back to the user's WhatsApp chat.

4. End-to-End System Architecture

Enterprise Dataflow and Isolation Boundary

```

[User via WhatsApp / Telegram / Slack] --> (Media Attachment: video.mp4, prompt: 'Check compliance')
    |
    v
[NVIDIA NemoClaw Security Envelope (OpenShell Sandbox)]
|-- OpenClaw Channel Gateway (Webhook Listener & Media Staging)
|-- Nemotron-3.5 Lightning (Intent Reasoning & Tool Orchestration)
+-- Local Execution Tools (extract_frames -> Cosmos NIM Client -> Deterministic Audit)
    |
    |--> Calls Local Docker: NVIDIA Cosmos 3 Reasoner NIM (http://localhost:8000/v1)
    +--> Validates Timestamps via Deterministic Engine (src/audit_engine.py)
    |
    +-- Formats Verdict (PASS/FAIL + Out-of-Order alerts + Evidence Log)
    |
    v
[Response to User Chat]: Markdown summary + Downloadable PDF Audit Report

```

5. Ambient Agent: Operational Patterns

To avoid high GPU idle power and resource contention, the ambient inspector operates on two selectable modes:

- > 1. On-Demand Conversational Mode: The agent runs in a passive listening state. It only invokes Nemotron and the GPU-intensive Cosmos NIM when a message or video is explicitly sent to the chat endpoint.
- > 2. Event-Driven Directory Watcher: The agent monitors an inbox folder (data/inbox/). Whenever a technician or surveillance feed saves a new video, the agent automatically detects it, matches it to the appropriate SOP, runs the inspection, and broadcasts alerts.

6. Security Envelope: OpenShell Guardrails

Industrial surveillance and assembly footage contain proprietary designs and worker privacy data. NemoClaw's OpenShell enforces strict kernel-level containment:

- > Filesystem Jail: Agent process can ONLY read from ./data/ and write to ./output/. Access to host root, system keys, or external drives is strictly denied.
- > Network Whitelisting: Inbound traffic is restricted to the chat gateway port (e.g. 8443 for webhooks). Outbound traffic is restricted to the local Cosmos NIM container (localhost:8000) and NVIDIA Nemotron API.
- > Prompt Injection Defense: User messages received from chat are sanitized before being passed to tool call arguments, preventing prompt injections from executing unauthorized shell commands.

7. Implementation Blueprint: Exposing Tools to OpenClaw

Your existing codebase (src/ingestion.py, src/cosmos_client.py, src/audit_engine.py) is registered as an OpenClaw agent tool inside NemoClaw:

Tool Definition: tools/compliance_tool.py

```
from src.ingestion import extract_frames
from src.cosmos_client import analyze_actions
from src.audit_engine import generate_compliance_report

def inspect_video_compliance(video_path: str, sop_path: str, output_name: str) -> dict:
    """Tool invoked by Nemotron when user requests a compliance audit."""
    frames = extract_frames(video_path, target_fps=1.0)
    with open(sop_path, 'r') as f: sop_data = json.load(f)
    observations = analyze_actions(frames, sop_data)
    obs_file = f'output/{output_name}_obs.json'
    report_file = f'output/{output_name}_report.json'
    with open(obs_file, 'w') as f: json.dump(observations, f)
    generate_compliance_report(sop_path, obs_file, report_file)
    with open(report_file, 'r') as f: return json.load(f)
```

8. Connecting WhatsApp and Chat Platforms

OpenClaw provides plug-and-play channel connectors. Here are the three primary connection methods:

- > A. WhatsApp Business Cloud API (Enterprise): Uses an official Meta Developer Account, Phone Number ID, and Permanent System Token. OpenClaw registers a webhook endpoint (HTTPS) to receive message payloads and media.
- > B. Twilio WhatsApp Gateway: Quickest setup with existing enterprise SMS/messaging accounts. OpenClaw binds via TWILIO_ACCOUNT_SID and TWILIO_AUTH_TOKEN.
- > C. Telegram Bot (Fastest for Prototyping): Requires only a single bot token from @BotFather. Supports instant video and file uploads up to 2GB with zero webhook SSL configuration.

9. Hardware and Deployment Considerations

- > Local Cosmos Container: Keep NVIDIA Cosmos 3 Reasoner NIM running in its local Docker container on your GPU (e.g. H100, A100, or RTX workstation) mapped to port 8000.
- > Nemotron 3.5 Cloud API: Using Nemotron via NVIDIA hosted cloud (integrate.api.nvidia.com) ensures zero extra VRAM contention on your local machine.
- > OpenShell Network Bridge: Ensure your Docker daemon allows bridge network communication between the OpenShell container and the Cosmos NIM container.